

Data Science & Machine Learning — Professional Masterclass

Turn raw data into reliable models, insights and deployable ML systems.

This is an original, project-first professional course manual created for educational and commercial course-development use.

The material is designed to be taught, adapted, branded and expanded. It emphasizes practical engineering, portfolio projects, assessments and production thinking rather than certificate collection. Technology names are used for educational context; specific provider pricing, product limits and interfaces should be verified before publication because they change over time.

Course Architecture

- 7 large modules with lessons, practical exercises, assessments and projects.
- Recommended delivery: 10–14 weeks at 6–10 hours per week, or self-paced.
- Primary outcome: a portfolio-ready capstone plus reusable professional skills.
- Suggested teaching method: concept → demonstration → guided lab → independent project → assessment.

Module Roadmap

1	Python, SQL and Data Foundations
2	Statistics and Experimental Thinking
3	Supervised Machine Learning
4	Unsupervised Learning and Deep Learning
5	ML Pipelines, Feature Stores and MLOps
6	Applied AI, Generative Models and Responsible ML
7	Capstone: End-to-End Data Product

Module 1 — Python, SQL and Data Foundations

Learning objectives

- Python for data work
- NumPy arrays
- Pandas dataframes
- SQL querying
- Data types and missing values
- Exploratory analysis
- Data quality

Core lesson

Data science begins with trustworthy data. Python provides the main programming environment, while SQL remains essential for retrieving and transforming data stored in databases.

Learn arrays, tables, indexing, joins, grouping, aggregation and transformations. Always inspect types, missing values, duplicates and suspicious ranges before modeling.

Exploratory data analysis is not simply making charts. It is a process of forming questions, testing assumptions and discovering structure. Compare distributions, segment by meaningful categories and inspect relationships between variables.

Data quality should be documented. Record where data came from, what each field means, how missing values are represented and which transformations were applied. This becomes especially important when datasets are reused months later.

Avoid data leakage from the beginning. Any information unavailable at prediction time must not enter the model features. Split data according to the real prediction scenario, especially when time or users are involved.

Practical lab

- Clean a messy CSV dataset.
- Write SQL queries using joins and aggregations.
- Produce an EDA report.
- Create a data dictionary.
- Identify and fix a potential leakage issue.

Assessment

Assessment: Explain why data leakage can produce excellent test scores and poor real-world performance.

Module project

Project: Data Quality & EDA Report — clean, profile and analyze a realistic dataset with reproducible notebooks.

Module 2 — Statistics and Experimental Thinking

Learning objectives

- Descriptive statistics
- Probability basics
- Sampling
- Confidence intervals
- Hypothesis testing
- Correlation vs causation
- A/B testing

Core lesson

Statistics gives data science a language for uncertainty. Mean, median, variance and quantiles summarize distributions, but the right statistic depends on the data and the question.

Probability helps reason about events and uncertainty. Understand conditional probability, independence and expected value at a practical level.

Sampling matters because a dataset is often a sample of a larger population. Sampling bias can invalidate otherwise sophisticated analysis. Ask who is represented, who is missing and how observations were collected.

Confidence intervals communicate uncertainty around an estimate. Hypothesis tests can help evaluate whether an observed difference is compatible with a null hypothesis, but they do not automatically prove causality or practical importance.

For A/B testing, define the primary metric and guardrails before seeing the result. Randomization protects against many confounders. Consider sample size, statistical power, multiple comparisons and practical effect size.

Correlation is not causation. A relationship may result from a third variable, reverse causality or selection effects. Good analysis combines statistical evidence with domain reasoning.

Practical lab

- Calculate descriptive statistics for several segments.
- Construct a confidence interval.
- Design an A/B test with a primary metric.
- Identify confounding variables in a case study.
- Interpret an effect size in business terms.

Assessment

Assessment: Design an experiment and explain its hypothesis, randomization, metric, sample and decision rule.

Module project

Project: Experiment Analysis — evaluate a simulated product experiment and produce a decision-ready report.

Module 3 — Supervised Machine Learning

Learning objectives

- Regression
- Classification
- Feature engineering
- Train/validation/test splits
- Cross-validation
- Metrics
- Model interpretation

Core lesson

Supervised learning maps input features to known targets. Regression predicts continuous values; classification predicts categories or probabilities.

Start with a simple baseline. A baseline establishes whether the model provides value beyond a naive rule. Then compare progressively more capable models.

Feature engineering can have large effects on model quality. Use domain knowledge to create meaningful representations, but ensure features are available at prediction time.

Separate training, validation and test data. Use cross-validation on training data when appropriate. Keep the final test set untouched until model selection is complete.

Choose metrics based on the business problem. Accuracy can be misleading for imbalanced classes. Precision, recall, F1, ROC-AUC and PR-AUC answer different questions. For regression, MAE and RMSE have different sensitivities to large errors.

Interpretability matters when users need to understand predictions. Use feature importance or local explanations carefully; an explanation method is not necessarily a causal explanation.

Practical lab

- Build a regression baseline.
- Build an imbalanced classification model.
- Compare at least three metrics.
- Use cross-validation.
- Write an error analysis instead of only reporting a score.

Assessment

Assessment: Select metrics for fraud detection, medical screening, sales forecasting and customer churn, and justify each choice.

Module project

Project: Predictive Model — complete supervised ML pipeline with baseline, feature engineering, evaluation and error analysis.

Module 4 — Unsupervised Learning and Deep Learning

Learning objectives

- Clustering
- Dimensionality reduction
- Anomaly detection
- Neural network foundations
- PyTorch concepts
- Training loops
- Overfitting and regularization

Core lesson

Unsupervised learning works without a target label. Clustering can reveal groups, dimensionality reduction can help visualization, and anomaly detection can identify unusual observations.

Neural networks are layered functions trained by gradient-based optimization. Understand tensors, parameters, forward passes, loss functions, backpropagation and optimizers conceptually before relying on high-level libraries.

Overfitting occurs when a model learns training-specific patterns that do not generalize. Use appropriate data splits, regularization, early stopping and data augmentation where suitable.

Deep learning becomes especially useful for images, audio, language and other high-dimensional data. Hardware and model size can become significant, so experiment with small architectures first.

Always compare against simpler methods. A neural network is not automatically better than a tree-based model for tabular business data.

Practical lab

- Cluster a customer dataset.
- Visualize high-dimensional data with dimensionality reduction.
- Build a small neural network.
- Plot training and validation loss.
- Diagnose an overfitting model.

Assessment

Assessment: Explain the difference between supervised and unsupervised learning and identify when deep learning is justified.

Module project

Project: Deep Learning Lab — train, evaluate and document a small neural model while demonstrating control of overfitting.

Module 5 — ML Pipelines, Feature Stores and MLOps

Learning objectives

- Reproducibility
- Data pipelines
- Feature pipelines
- Experiment tracking
- Model registry
- Batch vs real-time inference
- Monitoring

Core lesson

Machine learning systems are pipelines, not just models. Reproducibility requires tracking code, data versions, configuration, random seeds where relevant and model artifacts.

A data pipeline should transform raw inputs into validated training data. Add checks for schema changes, missing fields, unexpected distributions and duplicate records.

Experiment tracking records parameters, datasets, metrics and artifacts so models can be compared fairly. A model registry can store approved versions and deployment metadata.

Inference may be batch or real-time. Batch is often simpler and cheaper when predictions are not time-sensitive. Real-time inference needs an API, latency targets and availability controls.

Model monitoring should watch both system behavior and data behavior. Input distributions can drift. Prediction distributions can change. When labels eventually arrive, monitor actual performance too.

MLOps is about reducing the gap between an experiment that works on a laptop and a system that remains correct after deployment.

Practical lab

- Create a reproducible training pipeline.
- Track experiments and model versions.
- Add data validation checks.
- Deploy a batch prediction job.
- Design drift monitoring for one feature.

Assessment

Assessment: Design an ML lifecycle from raw data to training, approval, deployment and monitoring.

Module project

Project: MLOps Pipeline — reproducible training, experiment tracking, model packaging, inference and monitoring plan.

Module 6 — Applied AI, Generative Models and Responsible ML

Learning objectives

- LLM and ML integration
- Embeddings
- RAG basics
- Model evaluation
- Bias and fairness
- Privacy
- Responsible deployment

Core lesson

Modern data scientists increasingly work alongside generative AI. Traditional ML can classify, rank and forecast while LLMs can interpret unstructured text. Combining them often produces better systems than forcing one model to do everything.

Embeddings represent text or other data as vectors and can support semantic search, clustering and recommendation. RAG adds retrieved evidence to generation and is useful when information changes or must be traceable.

Evaluation must reflect the application. For generated text, measure factuality, relevance, groundedness and task completion. For predictive models, use task-specific metrics and monitor performance over time.

Responsible ML includes privacy, fairness, security and transparency. Avoid collecting data that is not necessary. Review whether a model's errors disproportionately affect particular groups. Provide human review where the consequences are high.

AI systems can amplify existing data problems. Document limitations and establish a process for feedback, correction and model updates.

Practical lab

- Build a semantic search prototype.
- Combine a classifier with an LLM summarizer.
- Create an evaluation rubric for generated outputs.
- Audit a dataset for missing or biased representation.
- Write a model card describing limitations.

Assessment

Assessment: Explain how traditional ML and generative AI can be combined in one product and identify risks.

Module project

Project: Applied AI Analytics System — predictive model plus semantic text capability with evaluation and responsible-AI documentation.

Module 7 — Capstone: End-to-End Data Product

Learning objectives

- Problem framing
- Data acquisition and quality
- EDA and modeling
- Evaluation
- Deployment
- Business impact
- Portfolio presentation

Core lesson

The capstone should solve a real decision problem. Begin with the decision, not the algorithm. Define who will use the output, what action it supports and what success means.

Build the data pipeline first. Validate inputs and document transformations. Perform exploratory analysis, establish a baseline, train models and conduct error analysis.

Deployment can be an API, scheduled job or interactive dashboard depending on the use case. Add authentication when data is sensitive. Log model version and prediction metadata so results can be traced.

Business impact should be quantified. A model with a high benchmark score may still have little value if it does not change decisions. Estimate time saved, revenue opportunity, reduced loss or improved customer experience.

Finish with a professional case study: problem, data, methodology, metrics, limitations, architecture, deployment and measured outcome. This becomes a strong portfolio artifact for internships and jobs.

Practical lab

- Choose a decision-oriented problem.
- Build a reproducible dataset pipeline.
- Train and compare models.
- Deploy predictions or insights.
- Present business impact and limitations.

Assessment

Assessment: Defend your model choice, evaluation design and deployment architecture to a technical review panel.

Module project

Capstone: End-to-End Data Product — data pipeline, analysis, model, evaluation, deployment and business-facing report.

Instructor & Commercial Packaging Guide

- Turn each module into 3–6 video lessons plus one guided lab.
- Provide a downloadable starter repository and a separate completed reference implementation.
- Add quizzes after each lesson and a practical assessment after each module.
- Keep project requirements stable while allowing learners to choose their own domain.
- For commercial delivery, add your own branding, instructor examples, screenshots, exercises and community/support layer.
- Avoid promising jobs, income or certification unless you actually provide the relevant accreditation or service.

Suggested Course Completion Certificate Criteria

- Complete at least 5 of 7 module projects.
- Pass the capstone assessment.
- Submit a working portfolio artifact and technical documentation.
- Demonstrate the ability to explain design decisions and limitations.

Final Portfolio Checklist

- Public or private demo URL where appropriate.
- Architecture diagram.
- README with setup and usage.
- Test/evaluation evidence.
- Security and limitations section.
- Short case study describing the problem and measurable result.